

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 4 — Un agente nel repository

Un file mai visto

Un file mai visto

Il riscaldamento del laboratorio: il docente per primo, poi voi

- Nel Modulo 4, Claude Code lavora nel repository: legge, agisce, controlla. Prima del laboratorio, un **riscaldamento**: un file che nessuno vi ha spiegato, messo in mano a Claude Code
- Il file: `src/Copertura/VerificaCopertura.php`, del **gestionale PHP** di CiaspoLink, il progetto del corso. Dice se un indirizzo è **coperto**, e con quale tecnologia
- Tre prompt: **spiegarlo** in dieci righe al massimo, con chi lo chiama, e scriverlo in `spiegazione.md`; un **test** del suo caso limite — la copertura disponibile proprio dal giorno della verifica — eseguito davvero; il **comando** che ha eseguito, e il suo **output**
- Venti minuti: il docente lo fa per primo, poi lo ripetete voi, ciascuno nel suo spazio di lavoro; poi la **verifica**, e il confronto

Che cosa dice Anthropic

Due frasi della documentazione di Claude Code: una messa alla prova, una da seguire

- La guida dei flussi di lavoro di Claude Code, sui test: «Claude examines your **existing test files** to match the style, frameworks, and assertion patterns already in use.»
- Il prompt del test chiede **che cosa** provare, non **come** scriverlo: se il test somiglia a quelli che il progetto ha già lo decide la **verifica**, sul file — «confermata» se le convenzioni valgono tutte, «non confermata in questa esecuzione» se no. È un esito su che cosa ha fatto Claude, non decide l'uscita della verifica
- La pagina delle best practice: «Whichever option you use, ask Claude for evidence: the **command it ran and its output**, or the screenshot.» E subito dopo: «"All tests pass" on its own tells you very little»
- È il terzo prompt, le **prove**: una raccomandazione da seguire, che la verifica non mette alla prova

Il docente, per primo

Tre prompt, uno alla volta, nella sessione del docente

```
# nello spazio di lavoro di M4-E01, pronto (passo 1 del README)
claude --model sonnet --effort medium
```

Claude Code su **Sonnet**, con effort **medium**: secondo la documentazione, Sonnet fa bene la maggior parte del lavoro di codice e costa meno di Opus. Le due opzioni valgono solo per questa sessione

il prompt 1, la spiegazione: il testo del README

```
Spiegami in dieci righe al massimo che cosa fa
progetti/gestionale-php/src/Copertura/VerificaCopertura.php e chi
lo chiama nel progetto. Poi scrivi la stessa spiegazione in
spiegazione.md, nella cartella in cui siamo, non dentro il
progetto.
```

Guardate che cosa **legge** prima di scrivere. Prima di approvare la scrittura di `spiegazione.md`, **leggetela**: è giusta?

Il docente, per primo

Tre prompt, uno alla volta, nella sessione del docente

il prompt 2, il test del caso limite: il testo del README

```
Aggiungi un test per il caso limite in cui la copertura diventa
disponibile proprio il giorno della verifica: disponibile_dal
uguale al giorno controllato, e l'indirizzo deve risultare
coperto. Poi esegui il test con questo comando, seguito dal
percorso del file di test, da solo, senza concatenarlo ad altri:
tools/php.sh php progetti/gestionale-php/vendor/bin/phpunit
-c progetti/gestionale-php
```

- Il prompt dice **che cosa** provare, non come scriverlo
- Leggete il test **prima di approvarne** la scrittura. Il comando dei test parte senza chiedervi niente: lo consente una regola `allow` del progetto
- Se Claude vi chiede di modificare `VerificaCopertura.php`: **No**. L'esercizio chiede un test, non una regola diversa

Il docente, per primo

Tre prompt, uno alla volta, nella sessione del docente

```
il prompt 3, le prove: il testo del README

Mostrami il comando che hai eseguito e il suo output.
```

- Confrontate l'output che vi mostra con quello che avete visto passare nella sessione
- **La registrazione del corso**, con `claude -p` senza approvazioni: tre prove, ognuna in uno spazio di lavoro nuovo

	prova 1	prova 2	prova 3
le sei convenzioni dei test che il progetto ha già	tutte «sì»	tutte «sì»	tutte «sì»
la frase della guida, decisa sul file del test	**confermata**	**confermata**	**confermata**
il test eseguito con `tools/php.sh`	passa: OK (6 tests, 19 assertions)	passa: OK (7 tests, 21 assertions)	passa: OK (6 tests, 19 assertions)
il test cade se il confine si sposta di un giorno	sì	sì	sì

Le tre prove: le righe che la verifica ha stampato, in tabella
Registrazione del corso, 28 ottobre 2026, Claude Code 2.1.312, Sonnet

- La frase della guida, decisa sul file del test: **confermata 3 su 3**

Ora voi

Gli stessi tre prompt, nel vostro spazio di lavoro: otto minuti

```
# dalla cartella esercitazioni: il passo 1 del README
tools/prepara_esercizio.sh M4-E01
cd ~/corso-claude-code/M4-E01
tools/php.sh composer install -d progetti/gestionale-php
claude --model sonnet --effort medium
# nella sessione: i tre prompt del README, uno alla volta
```

- La prima volta che aprite `claude` nello spazio di lavoro, Claude Code vi chiede se vi **fidate della cartella**: rispondete sì. Le regole `allow` del progetto, come quella del comando dei test, valgono **solo dopo**; le `deny`, subito
- **Traguardi**: `spiegazione.md` letta e approvata; il test letto, approvato, e **verde**; il comando e il suo output
- Una riga che unisce più comandi, con `cd …` `&&`: **No**, e chiedete un comando alla volta. Una modifica di `VerificaCopertura.php`: **No**. Il resto in «Se qualcosa va storto», nel README
- Il README: `m4_claude_code_agenti/s3_agente_nel_repository/m4_e1_un_file_mai_visto/M4-E01_file_mai_visto/`, nel repository delle esercitazioni
- L'impronta: una sessione su **Sonnet**, effort **medium**, tre prompt; nessun subagent chiesto; nella registrazione Claude ne ha avviati da sé in 2 sessioni su 3. La sessione conta sullo stesso limite di claude.ai; i test e la verifica girano sul vostro computer, **senza chiamate** al modello

Che cosa guardare

Nella vostra sessione, prima della verifica: tre domande, e una quarta

- **Legge prima di scrivere?** Prima del test, apre `VerificaCopertura.php` e i test che ci sono? Per Anthropic, Claude comincia ogni compito **leggendo il codice**
- **Esegue, o dice «dovrebbe funzionare»?** Claude si ferma quando il lavoro **sembra** finito; qui ha un controllo da eseguire, il comando dei test, che risponde «passa» o «fallisce»
- **Chiede prima di toccare una regola?** Se il test non passa, corregge il test o vi propone di cambiare `VerificaCopertura.php`? Una regola del dominio la decidete **voi**
- E la quarta: **l'agente non firma**. La spiegazione è utile solo se qualcuno la verifica: per Anthropic i test automatici aiutano a verificare la funzionalità, ma la **revisione di una persona** resta cruciale

Verifica

Il passo 5 del README: fuori dalla sessione, uno script controlla il lavoro

```
/exit                       # nella sessione di Claude Code
cd -                        # di nuovo nella cartella esercitazioni
tools/verifica.sh M4-E01    # 0 fatto, 1 non ancora, 2 non parte
```

- Legge lo spazio di lavoro con **git**, dal suo primo commit: conta anche il lavoro già in un commit
- `spiegazione.md` c'è e non è vuota: ne stampa le righe, e **non le giudica** — la legge una persona
- Un file di test **nuovo o cambiato** sotto `tests/`, con almeno un metodo di test nuovo, eseguito con `tools/php.sh` del repository, non con la copia dello spazio di lavoro: deve **passare**
- Le convenzioni dei test del progetto, una riga ciascuna: `declare(strict_types=1);` il namespace che rispecchia la cartella; `final class …Test extends TestCase`; le asserzioni `self::assert…`; i metodi `public function testInCamelCase(): void`; il database di prova `DatabaseDiProva`, nei test di integrazione. Poi la frase della guida, **decisa sul file**: non decide l'uscita
- Come **dato**: se il test cade quando il confine si sposta di un giorno, in una copia con `<=` cambiato in `<`; i file cambiati fuori da `tests/`
- Nella registrazione del corso, in tutte e tre le prove: «Fatto: spiegazione.md c'è e il test nuovo passa.»

Com’è andata

Il confronto, a voce

- A voce: ha letto prima di scrivere? Vi ha mostrato il comando e l’output?
- E il test ha **trovato** qualcosa? Se la verifica dice che non cade con il confine spostato di un giorno, il test passa, ma **non prova** il giorno stesso

Chiedete che legga prima di scrivere e che esegua invece di dire «dovrebbe funzionare»: la spiegazione dell’agente vale quando qualcuno la verifica, perché l’agente non firma.